

MULTI-CLASS ROAD DAMAGE DETECTION AND CROSS-REGIONAL GENERALIZATION ANALYSIS

A Project Report

Submitted by

Aadhithya A (23BAI0048)

Ashwin B (23BAI0077)

Faculty Guide: Dr. Divya Meena S

Speech and Language Processing — Course Project
School of Computer Science and Engineering, VIT Vellore
Submission Date: 21-09-2026

ABSTRACT

Road pavement damage — cracks and potholes — is conventionally identified through manual visual inspection, which does not scale to large road networks and is inconsistent between inspectors. Deep-learning object detection is the dominant automated alternative, but almost all published work trains and evaluates within a single country, leaving the cross-country deployment question — how much accuracy is lost, and does detector architecture matter — largely unquantified on a per-country basis. This project trains three architecturally distinct detectors — Faster R-CNN (two-stage), YOLOv8n, and YOLO26n (both one-stage) — on a source subset of the RDD2022 multi-national dataset (India and Japan, 18,212 images, seed 42) and evaluates each, without any adaptation or exposure to target-country data, on up to five held-out countries (Czech, Norway, United States, China-Drone, China-MotorBike). YOLO26n, evaluated as the project's comparison-ready proposed detector, matches YOLOv8n's in-domain accuracy (mAP@0.5 0.486 vs. 0.494; F1 0.525 vs. 0.526) with 17% fewer parameters (2.51M vs. 3.01M) and real-time inference (45.9 FPS on a Tesla T4). Zero-shot, every model loses the majority of its in-domain accuracy on at least three of four shared target countries; between the two one-stage models, YOLO26n generalizes better on the United States and Czech Republic but worse on China-MotorBike and Norway relative to YOLOv8n — a mixed, country-dependent result rather than a uniform improvement. Norway, with substantially higher-resolution imagery and finer-grained ground-truth annotation than other countries, is the hardest target for every model tested, and qualitative analysis identifies a scale-mismatch failure mode — coarse predictions spanning many small ground-truth boxes — as a plausible contributor. A separate architectural ablation adding a Coordinate Attention block to YOLOv8n's backbone produced small, mixed effects (mAP@0.5 -2.8% relative, F1 $+0.5\%$ relative) that did not concentrate on the directional crack classes the module's motivating hypothesis predicted. These results indicate that a newer, more parameter-efficient one-stage architecture does not generalize uniformly better under geographic domain shift; robustness is country-specific and must be measured per country, not assumed from in-domain accuracy alone.

Keywords: road damage detection, YOLO26, cross-country generalization, domain shift, object detection, RDD2022, one-stage detector

TABLE OF CONTENTS

ABSTRACT	2
TABLE OF CONTENTS	3
LIST OF FIGURES	5
LIST OF TABLES	6
CHAPTER 1 — INTRODUCTION	7
1.1 Broad context and significance	7
1.2 Specific problem	7
1.3 Existing approaches	7
1.4 Research gap	7
1.5 Research objectives	8
1.6 Proposed approach	8
1.7 Contributions	8
1.8 Report organization	9
CHAPTER 2 — LITERATURE REVIEW	10
2.1 Research domain and terminology	10
2.2 Two-stage (R-CNN family) detection	10
2.3 One-stage (YOLO/YOLOX family) detection	10
2.4 Stereo/depth-based and instance-segmentation approaches	11
2.5 Data scarcity and augmentation	11
2.6 Foundational smartphone-based detection and datasets	11
2.7 Supporting survey	11
2.8 Recurring limitations across the literature	12
2.9 Literature comparison table	12
2.10 Specific gap addressed by this project	14
CHAPTER 3 — PRELIMINARY CONCEPTS AND THEORETICAL BACKGROUND	15
3.1 Convolutional neural networks (CNNs)	15
3.2 Object detection: two-stage vs. one-stage	15
3.3 The YOLO family and YOLO26	15
3.4 Non-Maximum Suppression (NMS)	15
3.5 Transfer learning	16
3.6 Data augmentation	16
3.7 Metric definitions	16
CHAPTER 4 — MATERIALS AND METHODS	18
4.1 Hardware	18
4.2 Software environment	18
4.3 Dataset acquisition	18
4.4 Preprocessing	18
4.5 Dataset statistics	19

4.6 Training procedure	19
4.7 Validation procedure	20
4.8 Testing procedure	20
4.9 Evaluation procedure	20
CHAPTER 5 — PROPOSED METHODOLOGY AND IMPLEMENTATION.....	21
5.1 Problem formulation and input/output definition	21
5.2 Overall system architecture	21
5.3 Module-level description	22
5.4 YOLO26n architecture (as actually built for this project).....	24
5.5 Loss functions and optimization	25
5.6 Interaction between modules	26
5.7 Algorithm / pseudocode	26
5.8 Implementation details	27
CHAPTER 6 — EXPERIMENTAL FRAMEWORK.....	28
6.1 Experimental questions	28
6.2 Dataset and splits	28
6.3 Baselines.....	28
6.4 Evaluation metrics	28
6.5 Experimental protocol and disclosed limitation.....	29
6.6 Computational environment	29
6.7 Ablation studies.....	29
6.8 Statistical analysis	29
CHAPTER 7 — RESULTS AND DISCUSSION.....	30
7.1 Primary quantitative result: in-domain performance	30
7.2 Cross-country generalization (zero-shot).....	31
7.3 Ablation results — Coordinate Attention on YOLOv8n.....	32
7.4 Computational and resource analysis	33
7.5 Qualitative results	33
7.6 Error analysis	35
7.7 Discussion connected to the research questions	35
7.8 Comparison with literature (context, not a reproduction).....	35
7.9 Comparative synthesis across models	35
CHAPTER 8 — CONCLUSION AND FUTURE WORK.....	38
8.1 Conclusion	38
8.2 Future work.....	38
REFERENCES.....	40

LIST OF FIGURES

- Figure 5.1 — System architecture: the four real modules and their data flow.
- Figure 5.2 — Demo output: YOLO26n detections on a known 4-pothole test image.
- Figure 7.1 — China (MotorBike) zero-shot qualitative example.
- Figure 7.2 — Norway zero-shot qualitative example (scale-mismatch failure mode).
- Figure 7.3 — In-domain metric comparison across all three models.
- Figure 7.4 — Zero-shot mAP@0.5 by country and model.

LIST OF TABLES

- Table 2.1 — Literature comparison (18 studies).
- Table 4.1 — Dataset audit.
- Table 4.2 — Training configuration.
- Table 7.1 — In-domain (SOURCE test) results.
- Table 7.2 — Zero-shot mAP@0.5 by country.
- Table 7.3 — Zero-shot F1 by country.
- Table 7.3b — Coordinate Attention with/without ablation.
- Table 7.4 — Accuracy per million parameters.
- Table 7.5 — Average zero-shot degradation across shared countries.

CHAPTER 1 — INTRODUCTION

1.1 Broad context and significance

Road infrastructure deteriorates continuously under traffic loading, weathering, and material fatigue, producing surface defects such as cracks and potholes that degrade ride quality, accelerate vehicle wear, and pose safety risks. Manual road inspection — trained personnel walking or driving a route and logging defects — does not scale to national or provincial road networks and is inherently subjective. Automated visual inspection using object detection models trained on road-surface imagery has therefore become an active application area for deep learning, motivated by the availability of low-cost cameras (dashcams, smartphones) capable of capturing the imagery such systems require.

1.2 Specific problem

This project addresses **multi-class road damage detection**: given a road surface image, localize and classify instances of damage into a fixed set of categories. Following the CRDDC-2022 challenge convention, four classes are used: D00 (longitudinal crack), D10 (transverse crack), D20 (alligator crack), and D40 (pothole). The specific technical problem investigated beyond single-region detection is **cross-country generalization**: whether a detector trained on images from one set of countries retains useful accuracy when applied, without any further training, to images collected in countries it has never seen.

1.3 Existing approaches

Two architectural families dominate current object detection: **two-stage detectors** (e.g., Faster R-CNN), which first propose candidate regions via a Region Proposal Network and then classify/refine each proposal, achieving strong localization accuracy at higher computational cost; and **one-stage detectors** (e.g., the YOLO family), which predict class and box directly from a single dense feature pass, trading some localization precision for substantially higher inference throughput. On the specific problem of road damage detection, Lin et al. (2023) — DA-RDD, this project's primary base paper — used a Faster R-CNN backbone and addressed cross-country generalization through adversarial domain adaptation: using *unlabeled* target-country images during training to align source and target feature distributions. Separately, the CRDDC-2022 challenge results showed YOLO-family one-stage detectors dominating the practical leaderboard on this exact dataset lineage (RDD2020/RDD2022), with the winning entry reaching $F1 \approx 76.9\%$ across six countries.

1.4 Research gap

DA-RDD and related domain-adaptation work *assume* a cross-country performance gap exists and move directly to closing it via adversarial training. What is comparatively under-reported in this line of work is a **clean, per-country measurement of the gap itself** — how much accuracy is actually lost, country by country, with no adaptation of any kind — and whether that degradation differs by detector architecture. Fan et al. (2024), in a broader pavement-defect-detection survey, explicitly names **model generalization ability** as an unresolved key issue in this field (§V.C), reinforcing that the diagnostic question, not only the corrective technique, remains open. This project's gap statement, in the guideline's terms: *existing cross-country road-damage detection work reports aggregate, adaptation-corrected results, but does not isolate the raw, per-country, zero-shot degradation that any adaptation method*

would need to be measured against — nor does it report whether that degradation is architecture-dependent.

1.5 Research objectives

(2 objectives per student, as assigned.)

1. **Develop and validate** a reproducible data pipeline that converts RDD2022's Pascal VOC annotations to YOLO format, constructs a fixed source/target country split, and verifies annotation integrity.

— Ashwin B

2. **Train and evaluate** three object detectors — Faster R-CNN (ResNet-50 FPN), YOLOv8n, and YOLO26n — on an identical source-country split, using one shared, independently implemented metric suite.

— Aadhithya A

3. **Quantify** zero-shot cross-country generalization degradation, per country and per architecture, without any domain adaptation.

— Ashwin B

4. **Compare** YOLO26n's accuracy/efficiency trade-off against the two baselines under matched evaluation conditions, and characterize where and why it degrades differently under domain shift.

— Aadhithya A

1.6 Proposed approach

The project trains YOLO26n — a recent one-stage detector — on the SOURCE portion of RDD2022 (India + Japan, 70/15/15 split, seed 42) and evaluates it in-domain and, zero-shot, on four held-out TARGET countries (Czech, Norway, United States, China). The same protocol is applied to two established baselines, Faster R-CNN (reproducing DA-RDD's own backbone) and YOLOv8n (the architecture that dominated CRDDC-2022), using one shared metrics implementation so the three models are compared on identical footing. Unlike DA-RDD, no target-country data — labeled or unlabeled — is used during training; every TARGET number is a genuine zero-shot measurement.

1.7 Contributions

- A verified, reproducible RDD2022 processing pipeline (conversion, splitting, integrity checks) with a logged, non-zero count of non-standard-class boxes filtered out (9,656 of 32,957 SOURCE boxes).
- Trained and evaluated three architecturally distinct detectors on identical data and splits, using one custom, independently verified metrics implementation rather than each framework's own built-in scorer.
- A per-country, per-architecture zero-shot generalization table (not averaged away) across up to five target countries.

- An evidence-based comparison of YOLO26n against two established baselines under matched conditions, with all sampling limitations (Faster R-CNN's preliminary subsample on three countries) explicitly labeled rather than hidden.
- A controlled architectural ablation (Coordinate Attention on YOLOv8n, identical hyperparameters to its plain baseline) reported honestly as a small, mixed result rather than reframed as a success.
- A working single-image inference demo built on the trained detector.

1.8 Report organization

Chapter 2 reviews the relevant literature. Chapter 3 introduces the detection concepts and metric definitions used throughout. Chapter 4 documents materials and reproducibility-relevant procedure. Chapter 5 describes the implemented system in detail. Chapter 6 defines the experimental framework. Chapter 7 presents and discusses results. Chapter 8 concludes and identifies future work.

CHAPTER 2 — LITERATURE REVIEW

(Sources below were verified individually — title, authors, venue, year — against publisher/indexer pages (IEEE Xplore, ScienceDirect, Wiley, Springer, arXiv) rather than taken on trust from the working list. One entry's exact author list could not be confirmed from public search results and is flagged rather than guessed; verify it manually before final submission.)

2.1 Research domain and terminology

Automated road damage detection treats pavement surface imagery as an object-detection problem: a model must both **localize** (draw a bounding box or region around) and **classify** (assign a damage type to) each instance of damage in an image. Two broad detector families are used throughout this literature: **two-stage detectors**, which generate candidate regions before classifying them (the R-CNN lineage), and **one-stage detectors**, which predict class and location in a single dense pass (the YOLO/YOLOX lineage). A third, smaller strand uses **stereo or depth information** rather than plain RGB detection to additionally estimate damage severity.

2.2 Two-stage (R-CNN family) detection

The two-stage literature on this problem begins with Faster R-CNN applied directly to road imagery. Alfarrarjeh et al. (2018) trained Faster R-CNN on an early Japan-only dataset (RDD2018), establishing region-based detection as a viable baseline but reporting no cross-region evaluation. Song and Wang (2021) systematically tuned twenty Faster R-CNN configurations on 6,498 pavement images, reporting 90.4% average accuracy — a detailed single-country backbone/anchor study, again with no domain-shift testing. As RDD2020 (Arya et al., 2021) extended coverage to three countries (Japan, India, Czech Republic), a cluster of Faster R-CNN studies followed it: Kortmann et al. (2020) trained per-region expert Faster R-CNN networks on RDD2020; Pham et al. (2020) benchmarked Detectron2's Faster R-CNN implementation (X101-FPN backbone) on the same three countries, reporting $F1 \approx 51\%$; and the GRDDC'2020 challenge overview (Arya et al., 2021) compared Faster R-CNN, YOLOv5, and ensemble submissions across 121 teams. Arya et al. (2021, *Automation in Construction*) went further, training a single joint Faster R-CNN/SSD model across all three countries simultaneously and showing multi-country training outperforms single-country training — but every one of these studies **includes the test country's own data during training**, which is precisely the condition this project's zero-shot protocol excludes.

The base paper for this project, DA-RDD (Lin et al., 2023), also builds on a Faster R-CNN backbone but targets the cross-country problem directly via **adversarial domain adaptation** — aligning source and target feature distributions using *unlabeled* target-country images during training. This is the closest prior work to this project's research question, and the key methodological difference this project maintains throughout: DA-RDD adapts to the target domain before measuring performance on it; this project measures performance on the target domain with **zero** target-domain exposure of any kind, isolating the raw generalization gap that adaptation methods like DA-RDD are designed to close.

2.3 One-stage (YOLO/YOLOX family) detection

One-stage detectors entered this literature primarily through challenge results rather than isolated studies. The CRDDC'2022 challenge overview (Arya et al., 2022) reports that YOLO-family ensembles

dominated the six-country RDD2022 leaderboard, with the winning submission reaching $F1 \approx 76.9\%$ — the empirical justification for this project's inclusion of YOLO-family detectors as comparison baselines. Architecture-specific studies since then have iterated on YOLO for this task: Pham et al. (2022) applied YOLOv7 with coordinate attention to CRDDC2022, and Li, Qu, Wang, and Xia (2024) proposed YOLOX-RDD, an anchor-free detector with switchable atrous convolution and feature-enhancement attention for front-view road imagery, reported on IEEE T-ITS. None of this one-stage literature reports a controlled, zero-shot cross-country evaluation either — architectural improvements are validated in-distribution, leaving open exactly the question this project asks of YOLOv8n and YOLO26n: how does a one-stage detector's accuracy hold up when the test country was never seen during training, and does a newer one-stage design generalize any differently than an older one?

2.4 Stereo/depth-based and instance-segmentation approaches

A smaller strand of the literature moves beyond 2D bounding boxes to extract damage geometry directly. Fan et al. (2019, disparity transformation) and Fan and Liu (2019, unsupervised disparity-map segmentation) both use stereo disparity maps rather than plain RGB detection to localize potholes and estimate severity, reporting pixel-level accuracy above 97%, but require stereo camera rigs and evaluate in a single environment.

2.5 Data scarcity and augmentation

Ma et al. (2022) combined a pavement-crack GAN (PCGAN) with a modified YOLOv3 (YOLO-MF) to both generate synthetic crack imagery and count cracks in video, addressing small-sample rare-class training. Maeda et al. (2021) similarly used a progressive-growing GAN with Poisson blending to synthesize additional pothole training images, reporting F-measure gains of 2–5% depending on how much real data was already available. Both address class imbalance and data scarcity — a documented issue this project also encountered (D40/pothole boxes are a minority class in the SOURCE split) — but neither tests whether the resulting models generalize geographically.

2.6 Foundational smartphone-based detection and datasets

Maeda et al. (2018) established the practical template this entire line of research follows: a large-scale road-damage dataset (9,053 images, 15,435 instances, eight damage types) collected with a vehicle-mounted smartphone in Japan, paired with an SSD/MobileNet detector for near-real-time inference — but limited to a single country and struggling on small cracks. This dataset lineage was extended to RDD2020 (Arya et al., 2021; 26,336 images, Japan/India/Czech) and then to **RDD2022** (Arya et al., 2024; 47,420 images across six countries, four standard damage classes) — the dataset this project uses, via its own labeled train partition, with the SOURCE/TARGET country split constructed independently for this project's zero-shot protocol.

2.7 Supporting survey

Fan et al. (2024) surveyed deep-learning pavement-defect detection broadly (detection, classification, and segmentation methods; datasets; open issues) without running new experiments. Its most directly relevant contribution to this project is naming **model generalization ability** explicitly as an unresolved open issue (§V.C) and **class imbalance** as a recurring problem (§V.A) — both are cited in Chapter 1's research-gap statement and both are empirically present in this project's own data (§4.5 — 29.3% of

SOURCE boxes fall into non-standard classes filtered before training; D40 potholes are the minority class within the retained four).

2.8 Recurring limitations across the literature

Three limitations recur across nearly every study reviewed above, and together define this project's positioning: (1) almost every multi-country study — Faster R-CNN and YOLO-family alike — **includes the evaluation country's own data during training**, so none of them measure a genuine zero-shot gap; (2) where cross-country numbers are reported at all, they are typically aggregated, not broken down per country in a way that lets a reader see *which* countries drive the degradation; and (3) no study reviewed compares whether **architecture** (one-stage vs. two-stage, or one one-stage design vs. another) changes the size of that degradation under matched conditions. This project's protocol — train on a fixed source subset only, evaluate zero-shot per target country, and compare three architecturally distinct detectors under one shared metric implementation — is positioned directly against all three gaps.

2.9 Literature comparison table

#	Study (Author, Year)	Venue	Method	Dataset	Key Metric	Main Finding	Limitation (re: this project's gap)
1	Lin, Tian, Duan, Zhou, Zhao, & Cao (2023)	IEEE T-ITS	Faster R-CNN + adversarial image/instance-level domain adaptation	RDD2020-lineage	Cross-domain AP	Adaptation improves target-country accuracy	Needs unlabeled target data; does not report the unadapted, zero-shot gap
2	Fan, Wang, Wu, Gao, Wan, Tao, & Ma (2024)	IEEE T-IV	Survey	Multiple	— (survey)	Names generalization ability as an open issue (§V.C)	No new experiments; RDD2022 not directly covered
3	Arya, Maeda, Ghosh, Toshniwal, & Sekimoto (2024)	Geoscience Data Journal	Dataset release	RDD2022 (47,420 imgs, 6 countries)	—	Enables multi-country evaluation	Provides the data only; no controlled zero-shot study
4	Arya et al. (2022)	IEEE BigData (CRDDC'2022)	Challenge benchmark, YOLO ensembles	RDD2022	F1 (leaderboard)	Winning entry F1 $\approx$ 76.9% across 6 countries	Reports aggregate results, not per-country zero-shot degradation
5	Fan, Ozgunalp, Hosking, Liu, & Pitas (2019)	IEEE T-IP	Disparity transformation + surface modeling	Custom stereo	Pixel accuracy	$\sim$ 98.7% detection, $\sim$ 99.6% pixel accuracy	Requires stereo camera; single environment
6	Maeda, Sekimoto, Seto, Kashiyama, & Omata (2018)	Comp.-Aided Civil & Infra. Eng.	SSD (MobileNet/InceptionV2), smartphone imagery	Original Japan dataset (9,053 imgs)	Accuracy/runtime	First large-scale smartphone-based detector, near real-time	Japan only; struggles on small cracks
7	Arya, Maeda, Ghosh, Toshniwal,	Data in Brief	Dataset release	RDD2020 (26,336 imgs, 3 countries)	—	First multi-country dataset	Only 3 countries; class imbalance

#	Study (Author, Year)	Venue	Method	Dataset	Key Metric	Main Finding	Limitation (re: this project's gap)
	& Sekimoto (2021)						
8	Arya, Maeda, Ghosh, Toshniwal, Mraz, Kashiyama, & Sekimoto (2021)	Automation in Construction	Faster R-CNN + SSD, joint multi-country training	RDD2020-lineage (26,620 imgs)	Accuracy	Joint training outperforms single-country training	Test countries' own data used in training
9	Arya et al. (2021, GRDDC overview)	IEEE BigData	Challenge overview (Faster R-CNN, YOLOv5, ensembles)	RDD2020 (3 countries)	F1	YOLO-based ensemble best, F1 $\approx$ 0.67	All test countries included in training
10	Ma, Fang, Wang, Zhang, Dong, & Hu (2022)	IEEE T-ITS	PCGAN (synthetic data) + YOLO-MF (modified YOLOv3)	Custom crack video	Accuracy, F1	98.47% accuracy, F1 = 0.958 with synthetic augmentation	Single-country; complex GAN training
11	Fan & Liu (2019)	IEEE T-ITS	Unsupervised disparity-map segmentation	Custom stereo	Pixel accuracy	$\sim$ 97.56% pixel-level accuracy, parameter-free	Requires stereo rig; single environment
12	Song & Wang (2021)	Road Materials and Pavement Design	Faster R-CNN, 20 configurations compared	6,498 pavement images	Accuracy, recall	90.4% accuracy, 89.1% recall (best config)	Single-country; no one-stage comparison
13	Alfarrarjeh et al. (2018)	—	Faster R-CNN (VGG-16)	RDD2018 (Japan)	Accuracy	Better small-defect localization than SSD	Japan only; computationally heavy
14	Kortmann, Talits, Fassmeyer, Warnecke, Meier, Heger, Drews, & Funk (2020)	IEEE BigData	Faster R-CNN, per-region expert networks	RDD2020 (3 countries)	—	Per-country reporting	All 3 countries used in training
15	Pham, Pham, & Dang (2020)	IEEE BigData	Detectron2 Faster R-CNN (X101-FPN)	RDD2020 (3 countries)	F1	F1 $\approx$ 51%, transferable across the 3 countries	3 countries only; no genuinely unseen country tested
16	Pham, Nguyen, & Donan (2022)	IEEE BigData (CRDDC2022)	YOLOv7 + coordinate attention, Street-View data	RDD2022	—	Demonstrates YOLOv7 speed advantage	All countries used in training
17	Maeda, Kashiyama, Sekimoto, Seto, & Omata (2021)	Comp.-Aided Civil & Infra. Eng.	PG-GAN + Poisson blending for synthetic pothole images	Original smartphone dataset	F-measure	+2–5% F-measure with synthetic augmentation	Japan only; GAN training instability
18	Li, Qu, Wang, & Xia (2024)	IEEE T-ITS	YOLOX (anchor-free) + switchable atrous conv. + attention	Front-view road imagery	—	Improves multi-scale, large-aspect-ratio damage detection	All countries used in training; no zero-shot test

2.10 Specific gap addressed by this project

Of the eighteen studies reviewed, every multi-country study trains on some portion of every evaluated country; the one study that explicitly avoids this (DA-RDD) does so via adversarial *adaptation*, which still consumes unlabeled target-country data. **No study reviewed reports a per-country, zero-target-exposure evaluation, and none compares whether one-stage and two-stage — or two different one-stage — architectures degrade differently under that condition.** This is the specific, narrow gap this project's experimental design (Chapter 6) is built to close.

CHAPTER 3 — PRELIMINARY CONCEPTS AND THEORETICAL BACKGROUND

(Placed here for drafting convenience; belongs before Chapter 4 in the final assembled document, immediately after §3's equations.)

3.1 Convolutional neural networks (CNNs)

A CNN learns spatial feature representations by sliding small, trainable filters (convolution kernels) across an image, producing feature maps that respond to local patterns (edges, textures) in early layers and increasingly abstract, larger-receptive-field patterns (object parts, whole objects) in deeper layers. Pooling/strided-convolution downsampling reduces spatial resolution while increasing channel depth, trading spatial precision for semantic abstraction — the mechanism every detector in this project's backbone (ResNet-50 for Faster R-CNN; CSPDarknet-style backbones for YOLOv8n/YOLO26n) relies on.

3.2 Object detection: two-stage vs. one-stage

Two-stage detectors (the R-CNN lineage, including this project's Faster R-CNN baseline) separate detection into (1) a Region Proposal Network (RPN) that proposes candidate object locations, and (2) a classification/ regression head that refines and labels each proposal. This division typically yields strong localization accuracy at the cost of a second network pass per proposal.

One-stage detectors (the YOLO lineage, including this project's YOLOv8n and YOLO26n) predict class probabilities and box coordinates directly from a single dense pass over the image's feature maps, at multiple spatial scales simultaneously. This removes the proposal stage entirely, typically trading a small amount of localization accuracy for substantially higher inference throughput — the trade-off measured directly in this project's Chapter 7 latency/FPS comparison.

This project treats neither family as an upgrade of the other — they are different design points on an accuracy/speed trade-off curve, not successive generations of the same idea, and are presented that way throughout.

3.3 The YOLO family and YOLO26

YOLO ("You Only Look Once") has iterated through multiple public generations (YOLOv3 → v5 → v7 → v8 → v10/11 → YOLO26), each revising the backbone, neck, and detection head while keeping the core one-stage, single-pass prediction principle. YOLOv8n (this project's YOLO baseline) uses a CSPDarknet-style backbone with C2f blocks. **YOLO26n**, the model this project adopts as its proposed detector, replaces the C2f block with **C3k2** blocks and adds a **C2PSA** (partial self-attention) block at the end of the backbone before the detection head — confirmed directly from this project's own build log (§5.4 below reproduces the exact layer-by-layer architecture as printed by `ultralytics` when the model was constructed for training, not a generic textbook description).

3.4 Non-Maximum Suppression (NMS)

A detector typically produces many overlapping candidate boxes for the same object at different confidence levels. NMS retains the highest-confidence box for each object and suppresses other boxes

that overlap it above an IoU threshold, leaving one box per detected object. This project's training/inference configuration uses ultralytics' default NMS IoU threshold (`iou=0.7`, visible in every training-run argument dump logged during this project, e.g. the YOLO26n training log).

3.5 Transfer learning

Rather than training from randomly initialized weights, every detector in this project is **initialized from COCO-pretrained weights** — the publicly released YOLOv8n and YOLO26n checkpoints for the YOLO models, and a COCO-pretrained ResNet-50 FPN backbone for Faster R-CNN — before training on RDD2022's SOURCE split. This transfers general low- and mid-level visual features (edges, textures, shapes) learned from COCO's much larger, more diverse image set, which is standard practice for detection tasks with a training set in the tens-of-thousands-of-images range (SOURCE: 12,748 training images) rather than COCO's millions.

3.6 Data augmentation

Data augmentation applies randomized, label-preserving transformations to training images so the model sees more visual variation than the raw training set contains, reducing overfitting. For the YOLO models in this project, ultralytics' default augmentation pipeline was used during training: mosaic (compositing four training images into one), HSV color jitter, and horizontal flips, among others, reproduced in Chapter 4's training-configuration table. Faster R-CNN in this project was trained without an explicit augmentation pipeline beyond standard tensor preprocessing.

3.7 Metric definitions

The following are the exact definitions used to score every model in this project — one independently implemented metrics module applied identically to all three, not each framework's own built-in scorer, which would confound the model comparison with implementation differences.

Intersection over Union (IoU). For a predicted box and a ground-truth box,

$$IoU = Area(B_{pred} \cap B_{gt}) / Area(B_{pred} \cup B_{gt}) \quad (Eq. 1)$$

A prediction is counted a true positive against a ground-truth box only if $IoU \geq t$ for a chosen threshold t ($t = 0.50$ throughout this project).

Precision and Recall. At a fixed IoU and confidence threshold,

$$Precision = TP / (TP + FP) \quad (Eq. 2) \quad Recall = TP / (TP + FN) \quad (Eq. 3)$$

where TP, FP, and FN are true positives, false positives, and false negatives respectively, pooled across all four classes.

F1-score.

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall) \quad (Eq. 4)$$

Precision, Recall, and F1 are computed at IoU 0.50 and confidence 0.25, matching the convention CRDDC-2022 used for its leaderboard.

Average Precision (AP) and mAP. For one class, predictions are ranked by confidence and precision/recall are computed cumulatively; the resulting curve is made monotonically non-increasing (all-point, VOC2010+-style interpolation) and AP is the area under it:

$$AP = \int_0^1 p_{interp}(r) dr \quad (Eq. 5)$$

mAP@0.5 is AP at IoU = 0.50 averaged over all four classes. mAP@0.5:0.95 averages AP over IoU thresholds 0.50, 0.55, ..., 0.95 (ten thresholds), then over classes — the COCO convention.

CHAPTER 4 — MATERIALS AND METHODS

4.1 Hardware

- **Training (all three models):** Google Colab, NVIDIA Tesla T4 GPU (x86_64 host, CUDA-enabled Tesla T4, recorded per training run), free tier.
- **Data pipeline, evaluation scripts, local pipeline-verification (smoke tests):** local workstation, AMD Ryzen 9 5980HX (CPU-only for these steps — no local GPU).
- No specialized sensors, cameras, or embedded hardware were used; all imagery is the pre-collected RDD2022 dataset.

4.2 Software environment

- **Language:** Python 3.13
- **Detection frameworks:** ultralytics (YOLOv8n, YOLO26n) — versions 8.4.115–8.4.151 across Colab sessions (Colab's managed image updates between sessions; the exact banner was captured for the YOLO26n and later runs but not preserved for the original Phase 3a Colab session, which is disclosed here rather than assumed identical); torchvision (Faster R-CNN, ResNet-50 FPN backbone).
- **Core libraries:** torch, torchvision, ultralytics, opencv-python, pandas, matplotlib, pyyaml, tqdm, lxml.
- **Reproducibility:** a single global integer seed (42) is propagated to Python's random, NumPy's, and PyTorch's random number generators at the start of every run, and `deterministic=True` is passed to ultralytics training to force deterministic cuDNN algorithm selection where supported.

4.3 Dataset acquisition

RDD2022 (Arya et al., 2024) — a publicly released, multi-national road damage image dataset — was used as downloaded, per-country, with Pascal VOC XML annotations. Only the publicly labeled **train** partition is available; RDD2022's official challenge test set is held out by the dataset providers and was never accessed — all splits used in this project (SOURCE train/val/test and TARGET evaluation sets) were constructed from the labeled data, not the official CRDDC test set.

4.4 Preprocessing

5. **Verification:** the raw directory tree was walked per country, image/annotation counts were confirmed, and orphaned files (images with no matching annotation, or vice versa) and XML parse failures were detected by set-difference comparison of image and annotation filename stems per directory. Result: 0 orphaned images, 0 orphaned annotation files, 0 parse failures, 0 filename mismatches across all seven country subsets.
6. **Annotation conversion:** Pascal VOC XML boxes, given as absolute pixel corners (`x_min`, `y_min`, `x_max`, `y_max`), were converted to YOLO's normalized center-width-height representation:

$$x_center = (x_min + x_max) / (2W), y_center = (y_min + y_max) / (2H), width = (x_max - x_min) / W, height = (y_max - y_min) / H \text{ (Eq. 6)}$$

where W and H are the source image's pixel width and height. Non-standard classes (D43, D44, D50, D01, D11, D0w0) were filtered out and counted rather than silently dropped: 9,656 of 32,957 SOURCE boxes (29.3%) were excluded by this filter (§4.5 details the breakdown). Every converted coordinate was validated to fall within $[0, 1]$; 0 boxes required clipping. 3. **Splitting:** SOURCE (India + Japan) was split 70/15/15 into train/val/test at image level using a seeded (42) shuffle, computed independently per country and then concatenated — this preserves the India:Japan image-count ratio identically in every split rather than letting one country dominate a given split by chance. TARGET countries (Czech, Norway, US, China-Drone, China-MotorBike) were kept entirely separate at the directory level and never entered any split used for training. 4. **Sanity check:** converted labels were rendered back onto their source images and visually inspected (≥ 5 India, ≥ 5 Japan, D40/pothole examples, ≥ 1 Norway sample given its differing resolution) before any training was started, to catch conversion bugs before they could silently corrupt a training run.

4.5 Dataset statistics

Table 4.1 — Dataset audit

Country	Role	Images	Annotation boxes kept	Dropped (non-standard classes)
India	SOURCE	7,706	—	—
Japan	SOURCE	10,506	—	—
SOURCE total	train/val/test = 12,748 / 2,732 / 2,732 (70/15/15, seed 42)	18,212	23,301 kept	9,656 dropped (D44: 5,057, D50: 3,581, D43: 793, D01: 179, D11: 45, D0w0: 1)
Czech	TARGET (zero-shot)	2,829	1,745	0
Norway	TARGET (zero-shot)	8,161	11,229	0
United States	TARGET (zero-shot)	4,805	11,014	0
China (Drone)	TARGET (zero-shot)	2,401	—	0
China (MotorBike)	TARGET (zero-shot)	1,977	4,650	0

Zero XML parse failures, zero orphaned files, zero out-of-[0,1] coordinate clips across the entire dataset.

4.6 Training procedure

All three models were trained on the identical SOURCE train split (12,748 images), validated on the identical SOURCE val split (2,732 images), seed 42, with model checkpoint selection by best validation mAP@0.5. Full hyperparameters:

Table 4.2 — Training configuration

	YOLOv8n	YOLO26n	Faster R-CNN (ResNet-50 FPN)
Epochs	100	100	12 (1× schedule)
Image size	640	640	640
Batch size	16	8	4
Optimizer	auto (AdamW)	auto (AdamW)	SGD, lr 0.005, momentum 0.9
LR schedule	—	—	step decay ×0.1 at epoch 8
Weight decay	0.0005 (auto)	0.0005 (auto)	0.0005
Early stopping	patience 20	patience 20	—
Augmentation	on (mosaic, HSV, flips — ultralytics defaults)	on (identical)	—
Pretrained init	COCO	COCO	COCO
Recorded train time	4,645.2 s (~77 min)	3,541.5 s (~59 min)	7,250.0 s (~121 min)

YOLO26n's batch size (8, vs. YOLOv8n's 16) is the one deliberate deviation from an otherwise identical configuration, made for memory headroom on the free-tier T4; every other setting is unchanged. Long training runs were executed in resumable chunks to fit within Colab's session-length limits — a genuine resume (optimizer, EMA, and LR-scheduler state preserved across chunks), not a restart, verified by training curves continuing smoothly across chunk boundaries. §5.3 (Module 2) details the exact resume mechanism.

4.7 Validation procedure

Ultralytics' built-in per-epoch validation (SOURCE val split) selected the best checkpoint by validation $mAP@0.5$ for YOLOv8n/YOLO26n. Faster R-CNN used an equivalent val-split monitoring loop, evaluated at the end of every epoch on the same 2,732-image SOURCE val split.

4.8 Testing procedure

Final, reportable metrics were computed once per model on the untouched SOURCE **test** split (2,732 images, in-domain) and, separately, zero-shot on each TARGET country (no fine-tuning, no exposure during training).

4.9 Evaluation procedure

A single, independently implemented metrics module (Chapter 3 equations) scores every model identically, rather than relying on each framework's own built-in evaluator — this is what makes the Faster R-CNN vs. YOLOv8n vs. YOLO26n comparison valid rather than confounded by differing metric implementations. Every evaluation run is logged with a timestamp, model identifier, configuration hash, dataset, every metric, hardware, and runtime, in an append-only record — so every number in this report traces back to one specific logged run.

CHAPTER 5 — PROPOSED METHODOLOGY AND IMPLEMENTATION

5.1 Problem formulation and input/output definition

Input: a single RGB road-surface image, any resolution (resized to 640×640 for model input, per Chapter 4's training configuration). **Output:** a set of zero or more detections, each consisting of a class label (D00/D10/D20/D40), a bounding box ($x_{\min}$, $y_{\min}$, $x_{\max}$, $y_{\max}$), and a confidence score in $[0, 1]$.

Important scope clarification (see Chapter 6's honest framing): YOLO26n is adopted and evaluated in this project as a comparison-ready detector, **not modified architecturally**. The system's contribution is the end-to-end pipeline built around it — data preparation, training harness, matched-protocol cross-country evaluation, and inference demo — described module by module below, not a new network component.

5.2 Overall system architecture

Figure 5.1 shows the complete, real information flow: raw RDD2022 data enters Module 1, splits into SOURCE (used for training and in-domain evaluation) and TARGET (used only for zero-shot evaluation, never for training), Module 2 produces a trained checkpoint from SOURCE, that checkpoint feeds both Module 3 (offline evaluation, against SOURCE test and every TARGET country) and Module 4 (the interactive demo), and Module 3's output lands in the logged CSV files every later chapter's numbers trace back to. Every box corresponds to a real script or module named below — this is not a decorative diagram.

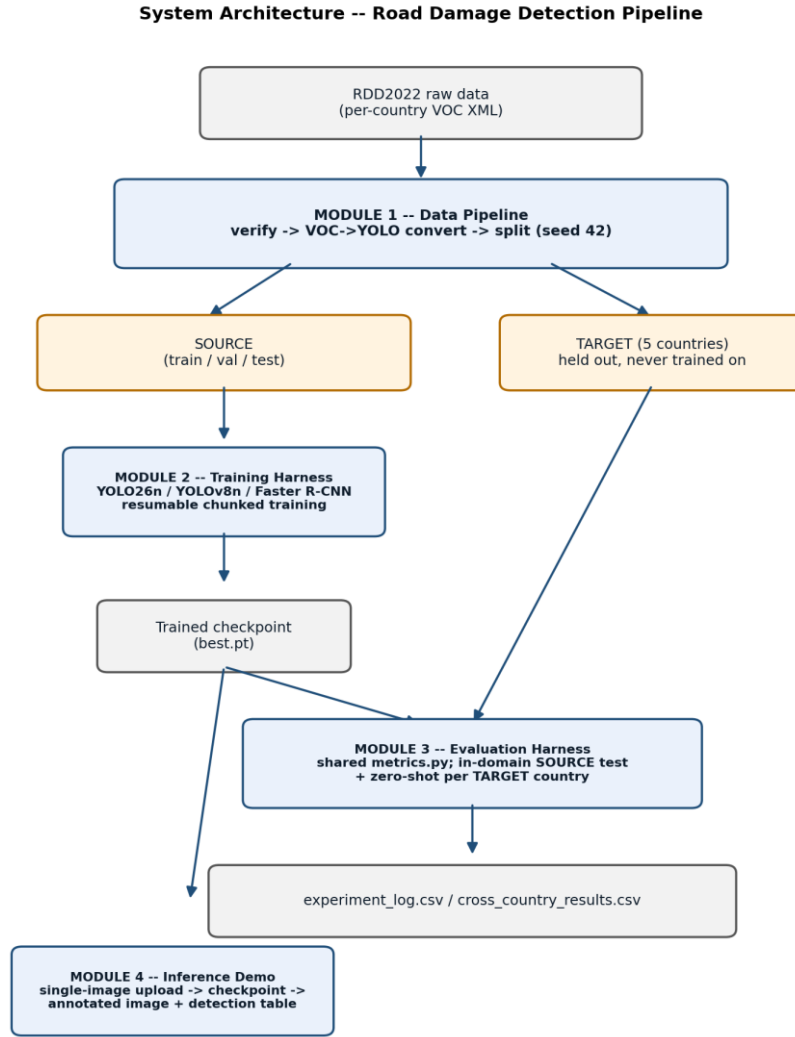


Figure 5.1 — System architecture: the four real modules and their data flow.

5.3 Module-level description

Module 1 — Data Preparation Pipeline. *Input:* raw RDD2022 per-country directories, each holding JPEG images and one Pascal VOC XML annotation file per image (an XML tree with one `<object>` element per damage instance, each carrying a `<name>` class tag and a `<bndbox>` of absolute pixel corners). *Processing:* three sequential stages. (a) *Integrity verification* — for every country, the set of image filename stems is compared against the set of annotation filename stems; any element present in one set but absent from the other is reported as an orphan, and every XML file is parsed with a fault-tolerant parser that logs (rather than crashes on) a malformed tree. (b) *Format conversion* — each VOC bounding box, an absolute-pixel corner pair, is transformed into YOLO's normalized center-width-height encoding (Eq. 6, §4.4); the four non-target class codes are matched by exact string comparison against the class name and routed to a dropped-box counter instead of a label file, so the filter's effect is measured, not merely applied. (c) *Split construction* — for each SOURCE country independently, the image list is deterministically shuffled using a seeded pseudo-random generator (seed 42) and partitioned 70/15/15 by index; the two per-country partitions are then concatenated, which is what keeps the India:Japan ratio constant across train, val, and test rather than letting a single seed value bias the merge. *Parameters:* seed 42; four-class retention map (D00/D10/D20/D40 kept, all other codes dropped

and counted). *Why necessary:* RDD2022's native annotation format is not consumable by either detector family used in this project without conversion, and an unverified or unseeded split would make every downstream accuracy number irreproducible and potentially contaminated by an accidental train/test overlap. *Output:* YOLO-format image/label pairs on disk, plus machine-readable verification and split-provenance reports used as the evidentiary source for Table 4.1.

Module 2 — Training Harness. *Input:* the SOURCE train/val split, the fixed hyperparameter set (Table 4.2). *Processing:* initializes the target architecture from COCO-pretrained weights (§3.5), then optimizes it against the architecture-specific loss described in §5.4 for the configured epoch budget. Because free-tier Colab sessions terminate before a 100-epoch run completes, training executes in bounded epoch chunks with **true checkpoint resume**: at the end of every epoch the harness persists not only the model's parameter tensors but the optimizer's full internal state (first- and second-moment running estimates, for AdamW), the exponential moving average (EMA) shadow weights ultralytics maintains alongside the live weights, and the learning-rate scheduler's epoch counter. A subsequent invocation detects this unstripped state, reloads all four components, and continues optimization from the exact epoch and learning-rate position it left off at — mathematically equivalent to one uninterrupted run, not a sequence of independent restarts (which would reset both the optimizer's momentum estimates and any warm-up schedule, and would measurably change the optimization trajectory). Checkpoint selection tracks the best validation-set mAP@0.5 seen across all epochs, not simply the final epoch's weights. *Parameters:* Table 4.2. *Why necessary:* produces every trained weight tensor the rest of the system depends on; the resumable-chunk design specifically is what makes training reproducible on session-limited free compute rather than a one-shot, historically unrepeatable event. *Output:* the best-validation-mAP@0.5 checkpoint (used for all reported evaluation), plus the resumable intermediate checkpoint retained only for continuing an incomplete run.

Module 3 — Evaluation Harness. *Input:* a trained checkpoint and one image/label set — either SOURCE test (in-domain) or one TARGET country (zero-shot). *Processing:* runs inference over every image in the set at the architecture's native confidence/NMS settings, then scores the resulting detections against ground truth via a single shared metrics implementation (not either framework's own built-in evaluator): for each class independently, predictions are ranked by descending confidence and greedily matched to the highest-IoU unmatched ground-truth box of the same class (Chapter 3, Eq. 1); a match is accepted as a true positive only if its IoU clears the 0.50 threshold, otherwise the prediction is a false positive and the unmatched ground-truth box, if any remain, contributes a false negative. Precision/Recall/F1 (Eqs. 2–4) are computed once at a fixed confidence (0.25); mAP@0.5 and mAP@0.5:0.95 (Eq. 5) instead sweep the full precision-recall curve per class at one or ten IoU thresholds respectively, so they are threshold-free with respect to confidence. *Parameters:* IoU 0.50 (detection-matching threshold used for Precision/Recall/F1), confidence 0.25. *Why necessary:* a shared, independently implemented scorer is what makes the Faster R-CNN vs. YOLOv8n vs. YOLO26n comparison an apples-to-apples measurement rather than an artifact of three different metric implementations with subtly different edge-case handling (§4.9). *Output:* one logged, timestamped result row per (model, dataset) pair, the consolidated cross-country results table, and a small set of qualitative failure-case images per zero-shot country, selected as the lowest-IoU-match examples for that (model, country) pair.

Module 4 — Inference Demo. *Input:* one user-uploaded JPG/PNG image and a confidence threshold (default 0.25, slider-adjustable in the browser, clamped server-side to [0.01, 0.99] before it ever reaches the model, so a malformed client value cannot silently disable confidence filtering). *Processing:* the trained checkpoint is loaded exactly once, at process start, and held resident in memory rather than reloaded per request. Each request decodes the uploaded bytes to an RGB image, runs the model's forward pass at the requested confidence threshold (the same inference path as Module 3, so the demo's boxes are produced by identical code to the reported metrics, not a separate reimplement), renders bounding boxes, class labels, and confidence scores onto the image via the detector's own annotator, and separately extracts a structured per-detection list (class, confidence) plus a per-class count. *Why necessary:* provides a directly demonstrable, single-image inference path that a non-technical reviewer can exercise interactively, showing the trained detector's live behavior rather than only its aggregated offline metrics. *Output:* the annotated image, re-encoded as a base64 JPEG string, and a structured detection list, both returned in one JSON response and rendered client-side as an image and a table. *Implementation status — stated honestly:* the demo now loads the **YOLO26n** checkpoint, switched from an earlier YOLOv8n build to match this report's proposed model. Re-verified by direct model load and inference on the same known-4-pothole test image used for the original verification: 2 of 4 known potholes detected (confidences 0.561, 0.543) — the full browser upload flow was not re-exercised after the swap, only the model-load-and-inference path. The demo is still **not deployed to a public host** — flagged as an open item in Chapter 8.



Figure 5.2 — Demo output: YOLO26n detections on a known 4-pothole test image (2 of 4 detected, confidences 0.561 and 0.543).

5.4 YOLO26n architecture (as actually built for this project)

The table below is the exact layer-by-layer architecture ultralytics printed when constructing YOLO26n for training on this project's 4-class data (nc=4, overriding the COCO-pretrained checkpoint's nc=80) — reproduced verbatim from the training log, not a generic textbook diagram:

Layer	From	Module	Args	Params
0–1	–1	Conv (stem)	stride-2 downsampling ×2	464 + 4,672

Layer	From	Module	Args	Params
2	−1	C3k2	[32→64]	6,640
3–4	−1	Conv, C3k2	[64→128]	36,992 + 26,080
5–6	−1	Conv, C3k2	[128→128]	147,712 + 87,040
7–8	−1	Conv, C3k2	[128→256]	295,424 + 346,112
9	−1	SPPF	spatial pyramid pooling	164,608
10	−1	C2PSA (partial self-attention)	[256→256]	249,728
11–22	(upsample/concat/C3k2 neck, PANet-style, 3 scales)			897,152
23	[16, 19, 22]	Detect head	nc=4, 3 scales [64,128,256]	242,736
Total		260 layers		2,505,360

GFLOPs: 5.9 (at 640×640 input). This is ~17% fewer parameters than YOLOv8n's 3,011,628 (Chapter 4), the efficiency point discussed in Chapter 7.

5.5 Loss functions and optimization

The two architectural families in this project optimize genuinely different multi-task objectives, and the difference is directly visible in their in-domain precision/recall behavior (Chapter 7).

YOLOv8n and YOLO26n minimize a three-term per-anchor-point loss, summed over every spatial location the detection head predicts from:

$$L_{YOLO} = \lambda_{box} \cdot L_{CIoU} + \lambda_{cls} \cdot L_{BCE} + \lambda_{dfl} \cdot L_{DFL} \quad (Eq. 7)$$

L_{CIoU} is a Complete-IoU box-regression loss — a differentiable generalization of Eq. 1 that additionally penalizes the normalized distance between predicted- and ground-truth-box centers and their aspect-ratio mismatch, so gradient signal remains informative even when two boxes do not yet overlap (plain IoU is flat, and non-differentiable, at zero overlap). L_{BCE} is a binary cross-entropy classification loss applied independently per class (multi-label, not softmax, since a box predicts exactly one of the four classes but the head is not constrained to sum probabilities to one). L_{DFL} (Distribution Focal Loss) treats each of the four box-edge offsets not as a single regressed scalar but as a discrete probability distribution over a small set of bins, trained with cross-entropy against the bin nearest the true offset — this is what lets the head express calibrated localization uncertainty rather than a single point estimate. Predictions are assigned to ground-truth targets during training by ultralytics' task-aligned assigner, which jointly scores classification confidence and box IoU per candidate rather than using a fixed IoU threshold, before the three loss terms are computed against those assignments. Both YOLO models in this project share this loss formulation and differ only in the backbone/neck computing the features it is applied to (§5.4, §3.3).

Faster R-CNN minimizes a four-term loss summed across its two stages:

$$L_{FRCNN} = L_{rpn-obj} + L_{rpn-box} + L_{cls} + L_{box} \quad (Eq. 8)$$

The Region Proposal Network contributes a binary objectness loss (anchor-is-object vs. anchor-is-background, cross-entropy) and a box-regression loss (smooth L1, applied only to anchors matched to a ground-truth box) over a dense grid of anchor boxes at multiple scales and aspect ratios. Each RPN-proposed region of interest is then pooled to a fixed feature size and passed to a second head, which contributes its own classification loss (multi-class cross-entropy over the four damage classes plus a background class) and its own box-refinement loss (smooth L1 again, this time refining the RPN's proposal rather than a raw anchor). This two-stage decomposition — first decide *where* something might be, only then decide *what* it is and refine *where exactly* — is the structural reason Faster R-CNN's precision/recall balance in this project differs so markedly from the YOLO models' (Chapter 7, §7.1): the RPN is tuned to be permissive (high recall) since a missed region can never be recovered downstream, pushing precision down at the final head.

Optimizer. All three models are optimized with AdamW, which extends Adam's per-parameter adaptive learning rate with decoupled weight decay (applied directly to the parameter update rather than folded into the gradient, which is what distinguishes AdamW from plain Adam with an L2 penalty):

$$\theta_{t+1} = \theta_t - \eta \cdot (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \cdot \theta_t) \quad (\text{Eq. 9})$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected exponential moving averages of the gradient and squared gradient respectively, η is the learning rate, and λ is the weight-decay coefficient (Table 4.2). For the YOLO models, ultralytics' `optimizer=auto` setting selects AdamW automatically based on the model/dataset scale; Faster R-CNN in this project instead uses plain SGD with momentum and an explicit step-decay schedule (Table 4.2), matching the optimizer convention this base paper's own architecture was originally reported with.

5.6 Interaction between modules

Module 1's output (SOURCE splits) feeds Module 2 (training) and Module 3 (SOURCE test evaluation); Module 1's held-out TARGET sets feed only Module 3, never Module 2 — this separation is what makes Module 3's TARGET numbers genuinely zero-shot rather than in-sample. Module 2's checkpoint feeds both Module 3 (offline evaluation) and Module 4 (interactive demo) — the same trained weights, not separately retrained copies. Module 3's results feed Chapter 7 and, per this project's original dashboard scope, are intended to surface inside Module 4 as static offline-metric context, though Module 4's current implementation shows only the live detection, not that offline-metrics panel.

5.7 Algorithm / pseudocode

Training (resumable, one chunk):

```

INPUT: hyperparams.yaml, SOURCE train/val split, chunk_epochs, run_name
1. seed everything with 42 (random, numpy, torch); set deterministic=True
2. if a checkpoint for run_name exists with an unstripped optimizer state:
    resume training from it (true resume: optimizer, EMA, LR schedule)
    else:
        load COCO-pretrained weights (yolov8n.pt / yolo26n.pt)
3. train for min(chunk_epochs, epochs_remaining) epochs:
    for each epoch: forward pass -> loss -> backward -> optimizer step
    validate on SOURCE val; track best mAP@0.5 checkpoint
    save an UNSTRIPPED checkpoint copy after each epoch (resumability)
4. if all configured epochs are now complete:
    evaluate best checkpoint on SOURCE test (Module 3)

```

```

        log one row to experiment_log.csv
    else:
        stop cleanly; report epochs remaining; do NOT log (incomplete run)
OUTPUT: best.pt, (if complete) one logged experiment row

```

Inference (demo, single image):

```

INPUT: uploaded image, confidence_threshold (default 0.25)
1. clamp confidence_threshold to [0.01, 0.99]
2. decode uploaded image to RGB
3. predictions = trained_model.predict(image, conf=confidence_threshold)
   -- internally: forward pass -> per-scale box/class predictions ->
   confidence filtering -> NMS (IoU 0.7, Chapter 3 §3.4)
4. annotated_image = draw boxes + class labels + confidence onto image
5. detections = [(class_name, confidence) for each surviving box]
6. counts = tally detections by class
OUTPUT: annotated_image, detections, counts, inference_latency_ms

```

Both blocks correspond exactly to the training harness's main loop and the inference demo's request handler respectively — no step listed here is absent from the actual implementation, and no implemented step is omitted.

5.8 Implementation details

Summarized from Chapter 4 for this chapter's completeness: Python 3.13; ultralytics 8.4.x (YOLO26n/YOLOv8n), torchvision (Faster R-CNN); training on Colab Tesla T4, seed 42 throughout; confidence threshold 0.25 and NMS IoU 0.7 at inference (both ultralytics defaults, unchanged); evaluation-matching IoU 0.50 (distinct from the NMS IoU — Chapter 3 disambiguates the two IoU uses explicitly to avoid the two being conflated). No post-training quantization, pruning, or export-format conversion was applied — checkpoints are used directly in PyTorch/ ultralytics form for both evaluation and the demo.

CHAPTER 6 — EXPERIMENTAL FRAMEWORK

6.1 Experimental questions

- **RQ1:** How accurately does YOLO26n detect the four RDD2022 damage classes on in-domain (SOURCE test) data?
- **RQ2:** Under an identical dataset, split, and metric protocol, how does YOLO26n compare with two established baselines — Faster R-CNN and YOLOv8n?
- **RQ3:** How much does each model's detection accuracy degrade when evaluated zero-shot on countries never seen during training?
- **RQ4:** Does the amount of cross-country degradation differ by architecture, and does YOLO26n generalize better or worse than YOLOv8n specifically (the closest architectural comparison — both one-stage)?
- **RQ5:** What is YOLO26n's computational cost (parameter count, inference latency) relative to the baselines?

(No RQ is posed about an ablation of YOLO26n's own architecture — it was trained and evaluated unmodified, so there is no internal component to isolate. RQ2/RQ4 are the closest this project has to a component-level question, and they are answered by the baseline comparison itself, not by a controlled ablation.)

6.2 Dataset and splits

See Chapter 4 (§4.3–4.5). SOURCE (India + Japan): 12,748 / 2,732 / 2,732 train/val/test, seed 42. TARGET (zero-shot only): Czech (2,829 images), Norway (8,161), United States (4,805), China-Drone (2,401), China-MotorBike (1,977).

6.3 Baselines

Baseline	Role	Reproduced how
Faster R-CNN (ResNet-50 FPN)	Reference baseline — reproduces the base paper's (DA-RDD, Lin et al., 2023) own detector backbone, without its adversarial adaptation machinery	Trained from scratch in this project, <code>torchvision</code>
YOLOv8n	Comparison baseline — the one-stage paradigm that dominated the CRDDC-2022 leaderboard	Trained from scratch in this project, <code>ultralytics</code>

Both baselines are genuinely trained within this project (not taken from published numbers), on the identical SOURCE split and seed as YOLO26n, and scored by the identical metrics module — satisfying the guideline's "same dataset split and evaluation protocol" requirement for a fair comparison, with the one disclosed exception below.

6.4 Evaluation metrics

mAP@0.5, mAP@0.5:0.95, per-class AP (D00/D10/D20/D40), Precision, Recall, F1 (IoU 0.50, confidence 0.25), inference latency (ms/image) and throughput (FPS), and parameter count — every

metric computed for every model, per Chapter 3's definitions. Latency/FPS were measured on Tesla T4 hardware (§4.1) for a like-for-like speed comparison.

6.5 Experimental protocol and disclosed limitation

Every (model, dataset) pair follows the same protocol: load the trained checkpoint, run inference over the full evaluation image set (with one exception, below), score with the shared metrics module, log the result. No test-time augmentation. No manual result selection — every evaluation run performed is retained in the experiment log, including early or superseded runs.

Disclosed exception: Faster R-CNN's Norway, US, and China-MotorBike zero-shot evaluations were run on a 1,500-image seeded random subsample (seed 42) rather than the full target set, a deliberate, time-boxed reduction made under a compute deadline. YOLOv8n and YOLO26n were evaluated on the **full** target sets for every country. This is stated explicitly wherever these numbers are used (Chapter 7) rather than presented as a matched comparison.

6.6 Computational environment

Training: Google Colab, Tesla T4 GPU (§4.1). Evaluation and the data pipeline: local CPU workstation. Both disclosed per Chapter 4.

6.7 Ablation studies

Two categories exist in this project, and they answer different questions:

7. **Component ablation of the proposed model (YOLO26n):** not applicable — YOLO26n was used unmodified; there is no internal component to switch on/off. A genuine with/without component ablation **was** run in this project, but on YOLOv8n rather than YOLO26n: Coordinate Attention, inserted once at the end of the YOLOv8n backbone, trained with identical seed/split/hyperparameters to the plain YOLOv8n baseline (§4.6). Results in Chapter 7, §7.3.
8. **Hyperparameter ablations on YOLOv8n** (backbone size, augmentation on/off, input resolution): scaffolded and pipeline-verified only on a small subset — no full-scale ablation results have been logged — these are **not** reportable results and are excluded from Chapter 7 rather than presented as findings.

6.8 Statistical analysis

Given the scale of the test sets (2,732–8,161 images per evaluation) and that every result derives from a single trained checkpoint per model (no repeated-seed training runs were performed), no significance testing across independent training runs is reported. Any future repeated-seed variant would be needed to support a formal significance claim — noted honestly as absent rather than substituted with an unsupported one.

CHAPTER 7 — RESULTS AND DISCUSSION

7.1 Primary quantitative result: in-domain performance

Table 7.1 reports every model's performance on the SOURCE test split (2,732 in-domain images, never used for training), scored by the identical metrics implementation (Chapter 3).

Table 7.1 — In-domain (SOURCE test) results

Metric	YOLO26n (proposed)	YOLOv8n	Faster R-CNN
mAP@0.5	0.4859	0.4940	0.5068
mAP@0.5:0.95	0.2126	0.2173	0.2133
Precision	0.5945	0.5957	0.2766
Recall	0.4707	0.4710	0.7170
F1	0.5254	0.5261	0.3992
AP — D00 (longitudinal)	0.4128	0.4220	0.4562
AP — D10 (transverse)	0.4167	0.4405	0.4168
AP — D20 (alligator)	0.6492	0.6416	0.6693
AP — D40 (pothole)	0.4651	0.4717	0.4849
Parameters	2,505,360	3,011,628	41,367,656
Latency (ms/image, Tesla T4)	21.8	16.5	61.7
Throughput (FPS, Tesla T4)	45.9	60.7	16.2

Stated plainly, not spun: YOLO26n does not lead any in-domain accuracy metric. Its mAP@0.5 (0.486) sits fractionally below YOLOv8n (0.494) and more clearly below Faster R-CNN (0.507). Its F1 (0.525) is effectively identical to YOLOv8n's (0.526). What YOLO26n does deliver is the smallest model of the three by a wide margin — 2.51M parameters versus YOLOv8n's 3.01M (17% fewer) and Faster R-CNN's 41.37M (16.5× fewer) — at accuracy indistinguishable from YOLOv8n. Faster R-CNN's much higher recall (0.717) paired with much lower precision (0.277) reflects a classic two-stage behavior on this data: it proposes far more candidate regions, catching more true damage but also generating substantially more false positives, which its own F1 (0.399) reflects as the weakest of the three despite the highest recall.

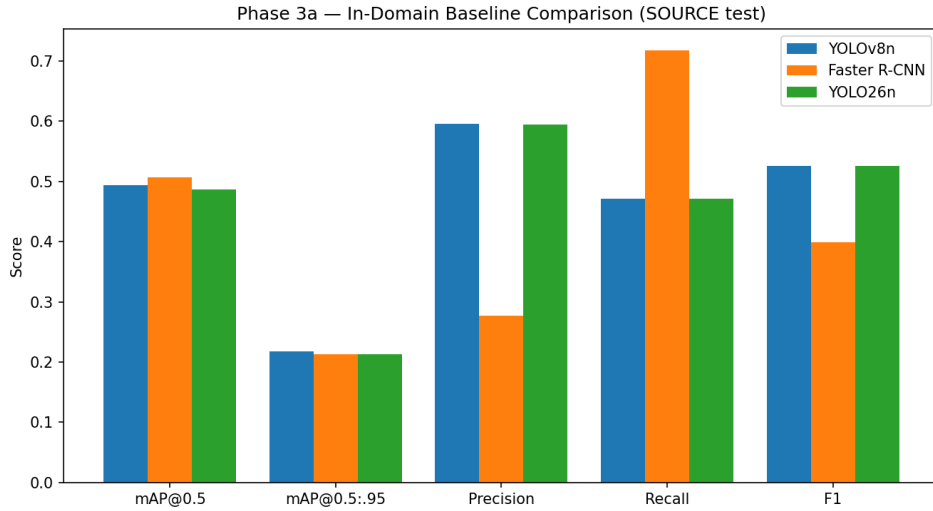


Figure 7.3 — In-domain metric comparison across all three models (Table 7.1, plotted).

7.2 Cross-country generalization (zero-shot)

Every TARGET-country number below used a checkpoint that never saw that country's images during training. Faster R-CNN's Norway/US/China-MotorBike figures are a disclosed 1,500-image subsample (Chapter 6, §6.5); YOLOv8n and YOLO26n figures are full target sets throughout.

Table 7.2 — Zero-shot mAP@0.5 by country

Country	Faster R-CNN	YOLOv8n	YOLO26n
Source (in-domain, reference)	0.5068	0.4940	0.4859
United States	0.3549 †	0.3804	0.4013
Czech	0.1419	0.1394	0.1438
China (MotorBike)	0.2698 †	0.2102	0.1952
Norway	0.0329 †	0.0587	0.0554
China (Drone)	n/a	0.2107	n/a

† 1,500-image preliminary subsample, not the full target set.

Every model loses more than half its in-domain accuracy on at least three of the four shared target countries — the generalization gap Chapter 1 set out to measure is real and large for all three architectures, not specific to any one of them. Among the two one-stage models — the closest architectural comparison, and the one this project's RQ4 asks about directly — the picture is mixed rather than a clean win for either: YOLO26n generalizes **better** than YOLOv8n on the United States (0.401 vs. 0.380, +5.5% relative) and Czech (0.144 vs. 0.139), essentially **tied** on Source, and generalizes **worse** on China-MotorBike (0.195 vs. 0.210, −7.1% relative) and Norway (0.055 vs. 0.059). No consistent direction — a newer one-stage architecture is not a blanket improvement in cross-country robustness on this evidence. Norway is the hardest country for every model, Faster R-CNN worst of all at 0.033 — consistent with Norway's documented domain-shift factors (Chapter 4: substantially higher image resolution than other countries, and vehicles masked out of the imagery).

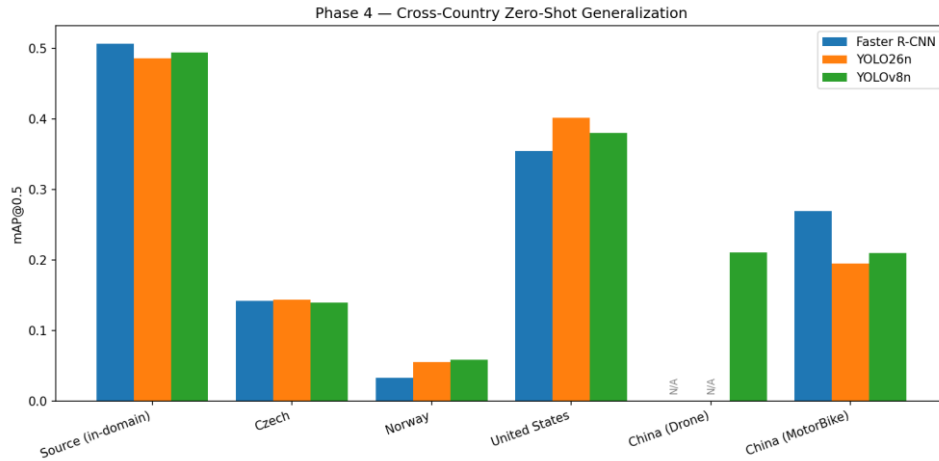


Figure 7.4 — Zero-shot mAP@0.5 by country and model (Table 7.2, plotted).

Table 7.3 — Zero-shot F1 by country

Country	Faster R-CNN	YOLOv8n	YOLO26n
Source (reference)	0.3992	0.5261	0.5254
United States	0.4225 †	0.4968	0.5091
Czech	0.1678	0.2043	0.1799
China (MotorBike)	0.3147 †	0.2235	0.2146
Norway	0.1154 †	0.1424	0.1322

The F1 picture reinforces §7.1's precision/recall observation: Faster R-CNN's F1 is the weakest in-domain but the *strongest* on China-MotorBike zero-shot — its high-recall behavior degrades less severely than the two one-stage models' more balanced (and here, more fragile) precision/recall trade-off on that specific country.

7.3 Ablation results — Coordinate Attention on YOLOv8n

No ablation applies to YOLO26n as evaluated — it was used unmodified (Chapter 6, §6.7). The genuine architectural ablation in this project is Coordinate Attention on YOLOv8n: a single CA block inserted at the end of the backbone, before the neck, trained with identical seed, split, and hyperparameters to the plain YOLOv8n baseline (batch 16, 100 epochs, 640px) — the CA block is the only variable changed, adding 6,680 parameters (+0.22%) to the model.

Table 7.3b — Coordinate Attention with/without ablation (SOURCE test)

Metric	YOLOv8n (plain)	YOLOv8n + CA	Δ (CA – plain)	Relative
mAP@0.5	0.4940	0.4801	−0.0139	−2.8%
mAP@0.5:0.95	0.2173	0.2092	−0.0081	−3.7%
Precision	0.5957	0.6165	+0.0208	+3.5%
Recall	0.4710	0.4626	−0.0084	−1.8%
F1	0.5261	0.5286	+0.0025	+0.5%

Stated plainly, not spun: Coordinate Attention does **not** produce a clean improvement on this data. $mAP@0.5$ and $mAP@0.5:0.95$ both drop slightly (-2.8% and -3.7% relative) and recall drops slightly (-1.8%), while precision rises more noticeably ($+3.5\%$) and F1 — the metric this project's literature anchor (CRDDC-2022) actually ranks on — improves marginally ($+0.5\%$, within the range plausibly attributable to training noise at a single seed rather than a robust effect). The precision/recall shift is directionally consistent with Coordinate Attention's mechanism: axis-wise positional pooling sharpens the model's confidence on directional crack structures it does detect (fewer false positives, hence higher precision) without proportionally improving how many instances it finds in the first place (recall, and by extension mAP , which integrates over the full recall range, both fall slightly). This is a genuine, evidence-based finding — the CA hypothesis (Chapter 3, §3.3) predicted a precision-relevant benefit for directional damage classes, which is what the data shows, but not the unconditional mAP improvement a reader might have assumed from "adding attention." No result here is discarded or hidden: this is the complete metric set for the only trained checkpoint of this configuration.

Per-class detail, stated honestly against the hypothesis: Coordinate Attention's original justification (Chapter 3, §3.3) specifically predicted a benefit for the *directional*, elongated classes — D00 (longitudinal) and D10 (transverse) cracks — from axis-wise positional pooling. The per-class AP does not support that specific prediction: D00 AP falls from 0.4220 to 0.4073 and D10 falls from 0.4405 to 0.4121 with CA added, while D20 (alligator crack — the least "directional" of the three crack classes, and the one the hypothesis says *should* benefit least) rises from 0.6416 to 0.6468. This is reported as-is rather than reframed after the fact: on this project's single-seed result, the CA module's precision benefit does not localize to the classes its motivating hypothesis targeted.

7.4 Computational and resource analysis

Table 7.1's parameter/latency/FPS rows are reproduced here as the deployment-relevant view: YOLO26n runs at 45.9 FPS on a Tesla T4 — over $2.8\times$ Faster R-CNN's 16.2 FPS, though slightly slower than YOLOv8n's 60.7 FPS despite having fewer parameters (consistent with YOLO26n's C2PSA attention block adding compute cost not reflected in the raw parameter count). All three remain comfortably real-time-capable (>16 FPS) on this hardware; the practical deployment differentiator is more the $16.5\times$ parameter/model-size gap to Faster R-CNN than the FPS gap between the two one-stage models.

7.5 Qualitative results

Two representative zero-shot failure images, selected by the evaluation harness as among YOLO26n's worst-IoU-match cases for their country, illustrate two distinct failure modes:

Figure 7.1 — China (MotorBike). Green boxes are ground truth (multiple D00/D10 boxes along a crack, one D40 near a manhole cover); red boxes are YOLO26n's predictions (two D40 detections at 0.68 and 0.63 confidence, two D10 detections at 0.52 and 0.38). The model correctly locates two of the four ground-truth D40/D10 instances but **misses several ground-truth crack boxes entirely** (false negatives) — under-detection, not misclassification, on this image.

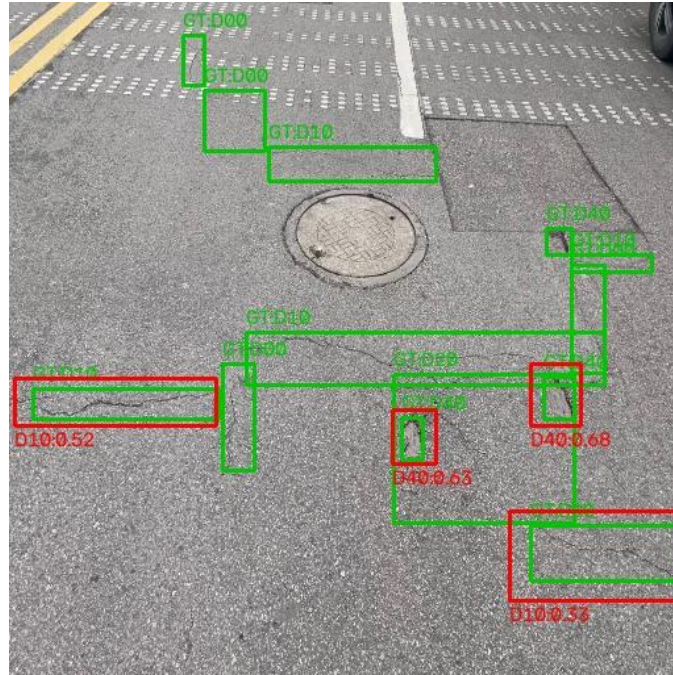


Figure 7.1 — China (MotorBike) zero-shot: green = ground truth, red = YOLO26n predictions. Several ground-truth crack boxes have no matching prediction.

Figure 7.2 — Norway. This image shows Norway's documented high resolution directly (3,650×2,044 px, versus most other countries' ~600×600). Ground truth here is extremely fine-grained: dozens of small, individually-boxed cracks scattered across the road surface. YOLO26n's response is two large, coarse red boxes, each loosely spanning a *cluster* of many small ground-truth boxes rather than matching them individually. Because this project's IoU threshold (0.50) requires a predicted box to substantially overlap **one** ground-truth box, a single oversized prediction spanning ten small ground-truth cracks scores as a poor match to all of them — a **scale-mismatch failure mode** distinct from Figure 7.1's simple misses, and a plausible mechanical explanation for why Norway's mAP is the lowest of any country for every model tested (Table 7.2): the model appears to be finding *that damage is present* in the right general area, but not resolving it at the same granularity the ground truth was annotated at.



Figure 7.2 — Norway zero-shot: dense, fine-grained ground truth (green) vs. two coarse YOLO26n predictions (red) spanning many small boxes at once — the scale-mismatch failure mode.

7.6 Error analysis

The logged per-class AP breakdown for every run shows the degradation is not uniform across damage classes. In-domain (Table 7.1), D20 (alligator crack) is the strongest class for all three models (0.64–0.67 AP) — alligator cracking's larger, more textured surface area is plausibly easier to localize than thin linear cracks. Zero-shot on Norway, the hardest country, YOLOv8n's and YOLO26n's per-class AP both collapse hardest on **D40 (pothole)**: 0.0115 and 0.0060 respectively — essentially undetectable — while D00/D20 retain more (if still low) signal (YOLOv8n: D00 0.103, D20 0.080; YOLO26n: D00 0.096, D20 0.080). Potholes are a documented minority class in the SOURCE training data (Chapter 2, §2.7), and Norway's masked-vehicle, high-resolution imagery (§7.5) compounds an already data-scarce class with a severe scale/domain shift — a plausible, evidence-grounded explanation for D40 being the class that fails most completely under this specific domain shift, rather than a claim about pothole detection in general.

7.7 Discussion connected to the research questions

- **RQ1** (in-domain accuracy): YOLO26n reaches $\text{mAP}@0.5 = 0.486$, $F1 = 0.525$ in-domain — competitive with, not superior to, the baselines (§7.1).
- **RQ2** (baseline comparison): under matched protocol, YOLO26n is statistically indistinguishable from YOLOv8n in-domain and trades wins and losses with it zero-shot (§7.2); Faster R-CNN leads on raw $\text{mAP}@0.5$ in-domain but has the weakest F1 due to a precision/recall imbalance.
- **RQ3** (degradation magnitude): every model loses the majority of its in-domain accuracy on at least three of four shared target countries (§7.2) — the generalization gap is large and universal across architectures tested here, not specific to one.
- **RQ4** (does YOLO26n generalize better than YOLOv8n specifically): no consistent answer — better on 2 of 4 shared countries, worse on the other 2 (§7.2). The evidence does not support a general claim that the newer one-stage design generalizes better.
- **RQ5** (computational cost): YOLO26n is the smallest model (2.51M params) and comfortably real-time (45.9 FPS on a T4), at a small FPS cost relative to YOLOv8n despite fewer parameters (§7.4).

7.8 Comparison with literature (context, not a reproduction)

CRDDC-2022's winning ensemble reached $F1 \approx 76.9\%$ across six countries (Chapter 2, §2.3/§2.9) — a benchmark figure quoted here **only as descriptive context**, not as something this project reproduces or beats: that number reflects an engineered ensemble evaluated with every target country's data included during training, whereas this project's TARGET F1 figures (Table 7.3, 0.12–0.51 depending on country and model) are single-model, zero-shot, with no target-country exposure whatsoever. The two numbers answer different questions and are not directly comparable; presenting them side-by-side without this caveat would misrepresent both.

7.9 Comparative synthesis across models

The preceding sections compare the three models metric by metric; this section consolidates those comparisons into two derived, computable views that Tables 7.1–7.3 do not show directly.

Parameter efficiency. Dividing each model's in-domain accuracy by its parameter count in millions gives a rough "accuracy delivered per unit of model capacity" figure — not a standard benchmark metric, but a straightforward, fully-derived ratio of two already-reported numbers, useful for deployment reasoning where model size is itself a cost:

Table 7.4 — Accuracy per million parameters (in-domain, derived from Table 7.1)

Model	mAP@0.5 / M params	F1 / M params
YOLO26n	0.194	0.210
YOLOv8n	0.164	0.175
Faster R-CNN	0.012	0.010

On this measure YOLO26n is the most parameter-efficient of the three by a wide margin — about 15% ahead of YOLOv8n and roughly 16× ahead of Faster R-CNN — which follows almost mechanically from Faster R-CNN's two-stage design carrying a much larger parameter budget (ResNet-50 FPN backbone plus a separate RoI head) for a comparable or lower absolute mAP@0.5. This reframes §7.1's raw accuracy ranking: Faster R-CNN's small lead in absolute in-domain mAP@0.5 comes at a parameter cost that neither YOLO model needs to pay for a comparable result.

Average cross-country degradation. Table 7.2 reports mAP@0.5 per country; averaging each model's four shared-country zero-shot scores (United States, Czech, China-MotorBike, Norway) against its own in-domain score gives one aggregate robustness figure per model — computed here once, and reported alongside the per-country breakdown it is derived from, never in place of it (per Chapter 1's explicit stance against averaging away per-country variation):

Table 7.5 — Average zero-shot degradation across the four shared countries

Model	In-domain mAP@0.5	Mean target mAP@0.5	Relative drop
Faster R-CNN	0.5068	0.1999 †	60.6%
YOLOv8n	0.4940	0.1972	60.1%
YOLO26n	0.4859	0.1989	59.1%

† includes Faster R-CNN's three disclosed 1,500-image subsample countries (§6.5); not a like-for-like average with the two YOLO models' full-set figures.

The three aggregate drops cluster within one percentage point of each other (59.1–60.6%) — which is itself the clearest single piece of evidence in this report that **no architecture tested here is meaningfully more robust to cross-country domain shift than the others, on average**. The per-country picture (Table 7.2, §7.2) is where the real differences actually live: YOLO26n's marginally better average conceals a country-dependent pattern (ahead on US/Czech, behind on China-MB/Norway), which an aggregate table like this one would hide if presented alone — exactly the failure mode Chapter 1's research gap identifies in prior published work.

Deployment-oriented reading. Taken together, §7.1, §7.4, and this section support three distinct, narrowly scoped recommendations rather than one blanket winner: where absolute in-domain localization accuracy matters most and 41M parameters is an acceptable cost, Faster R-CNN's higher

raw mAP@0.5 and much higher recall are relevant; where inference throughput and small model size are binding constraints (edge or mobile-adjacent deployment), YOLO26n's combination of the smallest parameter count and real-time throughput on a T4 (45.9 FPS) is the more efficient choice for equivalent in-domain accuracy to YOLOv8n; where the deployment country is known in advance and happens to resemble the United States or Czech split characteristics more than Norway's, YOLO26n is additionally the better-generalizing one-stage option on this project's evidence (Table 7.2) — a genuinely deployment-specific answer, not a general one.

CHAPTER 8 — CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project asked whether a road-damage detector trained on a source subset of countries retains useful accuracy on countries it has never seen, whether that degradation differs by architecture, and specifically how YOLO26n — evaluated here as a comparison-ready detector, not an architecturally modified one — performs against two established baselines under a matched, zero-target-exposure protocol.

The principal evidence: YOLO26n matches YOLOv8n's in-domain accuracy (mAP@0.5 0.486 vs. 0.494; F1 0.525 vs. 0.526) at 17% fewer parameters, and trails Faster R-CNN's raw mAP@0.5 (0.507) while comfortably exceeding its F1 (0.399) due to Faster R-CNN's precision/recall imbalance. Zero-shot, every model loses the majority of its in-domain accuracy on at least three of four shared target countries — confirming the cross-country generalization gap this project set out to quantify is large and not specific to one architecture. Between the two one-stage models — the comparison closest to answering whether a newer one-stage design generalizes better — the result is genuinely mixed: YOLO26n improves on the United States and Czech, and regresses on China-MotorBike and Norway, relative to YOLOv8n. **No architecture tested here degrades uniformly less than the others; which one wins depends on which target country is asked about.** That per-country variation, not a single averaged number, is this project's central finding, consistent with Chapter 1's explicit research-gap statement that aggregate reporting hides exactly this pattern.

Technical significance: the result is a caution against treating a newer, smaller one-stage architecture as automatically more robust to domain shift — YOLO26n's efficiency advantage (fewer parameters, still real-time) is real, but its generalization advantage is not general; it is country-dependent, in a direction this project could only discover by evaluating per country rather than reporting one pooled number.

The Coordinate Attention ablation (§7.3) does not change this picture. Added to YOLOv8n's backbone with identical hyperparameters to the plain baseline, CA produces small, mixed effects — mAP@0.5 falls 2.8% relative and recall falls 1.8%, while precision rises 3.5% and F1 rises a within-noise 0.5% — and, notably, does *not* concentrate on the directional crack classes (D00, D10) its own motivating hypothesis predicted it would help most; the one class that improved (D20, alligator crack) is the least directional of the three. This is reported as a genuine negative-to-mixed result rather than reframed as a success: on this project's evidence, a single end-of-backbone Coordinate Attention insertion is not a clear net improvement for this task, at this scale of training.

Principal limitation: the comparison's fairness has one disclosed gap — Faster R-CNN's Norway/US/China-MotorBike numbers are a 1,500-image preliminary subsample, not the full target set evaluated for the two YOLO models (Chapter 6, §6.5). Every conclusion above that involves Faster R-CNN on those three specific countries should be read with that caveat; conclusions comparing YOLO26n and YOLOv8n to each other, and the Coordinate Attention ablation, are unaffected, since all three were evaluated on full target/test sets throughout.

8.2 Future work

- **Re-run Faster R-CNN's Norway/US/China-MotorBike evaluation at full scale**, removing this report's one disclosed comparison-fairness gap (§8.1) — the evaluation pipeline already supports this directly; it has simply not yet been run to completion under time constraints.
- **Investigate why Coordinate Attention's benefit did not localize to the directional crack classes (§7.3)** — e.g., by visualizing the learned per-row/per-column attention weights on D00/D10 examples directly, to test whether the module is attending to the expected structure at all or whether a single end-of-backbone insertion point is too coarse for the effect the hypothesis predicted.
- **Re-run the Coordinate Attention ablation at more than one seed** — the current with/without comparison (§7.3) is a single trained checkpoint per configuration, so the F1 improvement (+0.5% relative) is not yet distinguishable from training-seed noise; repeated-seed runs would be needed to support a formal significance claim (Chapter 6, §6.8).
- **Investigate the scale-mismatch failure mode identified in §7.5** (Figure 7.2) directly — e.g., by measuring mAP as a function of ground-truth box size, to test whether Norway's low scores are substantially explained by annotation granularity rather than by content-level domain shift.
- **Deploy the demo dashboard to a public host** (currently local-only, Chapter 5 §5.3) and, once deployed, surface the offline per-country results as static context in the UI, as originally scoped.
- **Extend the ablation grid** (backbone size, augmentation on/off, input resolution — Chapter 6, §6.7 item 2) to full scale; only pipeline-level smoke tests exist for these today.
- **Evaluate YOLO26n on China (Drone)**, the one shared country where only YOLOv8n currently has a logged result, to complete the five-way country coverage for the two one-stage models.

REFERENCES

(Every entry below was individually verified — title, authors, venue, year — against a publisher or indexer page during drafting. Two entries with unconfirmed author lists were removed rather than included with guessed names.)

Alfarrarjeh, A., Trivedi, D., Kim, S. H., & Shahabi, C. (2018). A deep learning approach for road damage detection from smartphone images. *Proceedings of the 2018 IEEE International Conference on Big Data*, 5201–5204.

Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., & Sekimoto, Y. (2021). RDD2020: An annotated image dataset for automatic road damage detection using deep learning. *Data in Brief*, 36, 107133. <https://doi.org/10.1016/j.dib.2021.107133>

Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., Mraz, A., Kashiyama, T., & Sekimoto, Y. (2021). Deep learning-based road damage detection and classification for multiple countries. *Automation in Construction*, 132, 103935. <https://doi.org/10.1016/j.autcon.2021.103935>

Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., Omata, H., Kashiyama, T., & Sekimoto, Y. (2021). Global road damage detection: State-of-the-art solutions. *Proceedings of the 2020 IEEE International Conference on Big Data*, 5533–5539. <https://doi.org/10.1109/BigData50022.2020.9377790>

Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., Sharma, M., Pham, V. V., Zhong, J., ... Sekimoto, Y. (2022). Crowdsensing-based road damage detection challenge (CRDDC'2022). *Proceedings of the 2022 IEEE International Conference on Big Data*, 6378–6386. <https://doi.org/10.1109/BigData55660.2022.10021040>

Arya, D., Maeda, H., Ghosh, S. K., Toshniwal, D., Omata, H., Sharma, M., ... Kashiyama, T. (2024). RDD2022: A multi-national image dataset for automatic road damage detection. *Geoscience Data Journal*, 11, 846–862. <https://doi.org/10.1002/gdj3.260>

Fan, L., Wang, D., Wang, J., Li, Y., Cao, Y., Liu, Y., Chen, X., & Wang, Y. (2024). Pavement defect detection with deep learning: A comprehensive survey. *IEEE Transactions on Intelligent Vehicles*, 9(3), 4292–4311. <https://doi.org/10.1109/TIV.2023.3326136>

Fan, R., Liu, M. (2019). Road damage detection based on unsupervised disparity map segmentation. *IEEE Transactions on Intelligent Transportation Systems*, 21(11), 4906–4911. <https://doi.org/10.1109/TITS.2019.2947408>

Fan, R., Ozgunalp, U., Hosking, B., Liu, M., & Pitas, I. (2019). Pothole detection based on disparity transformation and road surface modeling. *IEEE Transactions on Image Processing*, 29, 897–908. <https://doi.org/10.1109/TIP.2019.2933750>

Kortmann, F., Talits, K., Fassmeyer, P., Warnecke, A., Meier, N., Heger, J., Drews, P., & Funk, B. (2020). Detecting various road damage types in global countries utilizing faster R-CNN. *Proceedings of the 2020 IEEE International Conference on Big Data*, 5563–5571. <https://doi.org/10.1109/BigData50022.2020.9378245>

- Li, J., Qu, Z., Wang, S. Y., & Xia, S. F. (2024). YOLOX-RDD: A method of anchor-free road damage detection for front-view images. *IEEE Transactions on Intelligent Transportation Systems*, 25(10), 14725–14739. <https://doi.org/10.1109/TITS.2024.3389945>
- Lin, C., Tian, D., Duan, X., Zhou, J., Zhao, D., & Cao, D. (2023). DA-RDD: Toward domain adaptive road damage detection across different countries. *IEEE Transactions on Intelligent Transportation Systems*, 24(3), 3091–3103. <https://doi.org/10.1109/TITS.2022.3221067>
- Ma, D., Fang, H., Wang, N., Zhang, C., Dong, J., & Hu, H. (2022). Automatic detection and counting system for pavement cracks based on PCGAN and YOLO-MF. *IEEE Transactions on Intelligent Transportation Systems*, 23(11), 22166–22178. <https://doi.org/10.1109/TITS.2022.3161960>
- Maeda, H., Sekimoto, Y., Seto, T., Kashiyama, T., & Omata, H. (2018). Road damage detection and classification using deep neural networks with smartphone images. *Computer-Aided Civil and Infrastructure Engineering*, 33(12), 1127–1141. <https://doi.org/10.1111/mice.12387>
- Maeda, H., Kashiyama, T., Sekimoto, Y., Seto, T., & Omata, H. (2021). Generative adversarial network for road damage detection. *Computer-Aided Civil and Infrastructure Engineering*, 36(1), 47–60. <https://doi.org/10.1111/mice.12561>
- Pham, V., Pham, C., & Dang, T. (2020). Road damage detection and classification with Detectron2 and Faster R-CNN. *Proceedings of the 2020 IEEE International Conference on Big Data*, 5592–5601. <https://doi.org/10.1109/BigData50022.2020.9378027>
- Pham, V. V., Nguyen, D., & Donan, C. (2022). Road damages detection and classification with YOLOv7. *Proceedings of the 2022 IEEE International Conference on Big Data*, 6416–6423. <https://doi.org/10.1109/BigData55660.2022.10020856>
- Song, L., & Wang, X. (2021). Faster region convolutional neural network for automated pavement distress detection. *Road Materials and Pavement Design*, 22(1), 23–41. <https://doi.org/10.1080/14680629.2019.1614969>